Leave Management System

Build & Refactor using Object-Oriented Programming in Java

Monthly Assignment – Module 2: Object-Oriented Programming
Software Development Batch 4 B'26 [Aug]

Name	Sai Krishna S.
Roll No / ID	<YOUR ROLL NO / ID>
Date of submission	06 October 2026
GitHub repository	`https://github.com/<YOUR-USERNAME>/<YOUR-REPOSITORY>`

Assignment Requirements Covered

The uploaded assignment brief asks for design before coding, a basic leave application, application of OOP concepts, a Manager change request, debugging/refactoring, Git practice, and the final Java source code, class diagram, README, sample output, repository link, OOP explanation and refactoring notes. The final package below is structured directly around those requirements. The supplied brief identifies the task as a Java OOP Leave Management System assignment.[1]

Requirement	Where covered in this submission
Java source code	Appendix A and `src/leave/`
Class diagram	Section 5 and `images/class_diagram.png`
README file	Section 13 and `README.md`
Sample output / screenshots	Section 10 and `images/console_output.png`
GitHub repository link	Cover table and Section 12
OOP concepts	Section 6
Refactoring notes	Section 8
Git practice	Section 12 and `docs/GIT_COMMIT_PLAN.md`

1 Choosing the Problem

A company needs a simple system to manage employee leave. Different employee types receive different leave rules: a full-time employee receives more annual leave than an intern, while a manager receives additional benefits. This is a suitable OOP problem because the same operation, such as applying for leave, can behave differently depending on the employee type.

The final program stores employee details, applies and cancels leave, checks the current balance, stores leave requests separately, and reports invalid operations through custom exceptions. The leave numbers are sample values for this assignment and are not intended to represent a real company policy.

2 Design Before Coding: Problem Analysis

The assignment explicitly requires the main objects, data, behaviours, classes and relationships to be identified before implementation. The final design is as follows.

[1] Source: uploaded assignment brief, pages 1–8.

Part	Description
Input	Employee ID, employee name, department, employee type, number of leave days, reason and request ID for cancellation
Process	Validate positive days, enforce role-specific per-request limit, check available balance, deduct balance, store request, and restore balance on cancellation
Output	Employee details, approved request details, updated balance and clear error messages for rejected requests

Leave rules used:

Type	Annual leave	Max/request	Notes
Full-Time	24	10	Base rules
Part-Time	12	5	Reduced entitlement and limit
Intern	6	2	Entry-level sample rule
Manager	30	15	24 base + 6 additional days; reuses Full-Time hierarchy

Main relationships: `Employee` is an abstract base class. `FullTimeEmployee`, `PartTimeEmployee` and `Intern` extend it. `Manager` extends `FullTimeEmployee`. An employee owns a list of `LeaveRequest` objects. `LeaveException` is the base checked exception for invalid leave operations.

3 Breaking the Problem into Smaller Tasks

1. **Design:** identify classes, data, behaviours and relationships; prepare the class diagram.
2. **Base class:** create `Employee` with employee identity, department, entitlement, balance and common leave operations.
3. **Leave requests:** extract request identity, days, reason and status into `LeaveRequest`.
4. **Inheritance:** create Full-Time, Part-Time and Intern employee types with their own limits.
5. **Change request:** add Manager by extending Full-Time behaviour.
6. **Error handling:** use custom exceptions for invalid requests and insufficient balance.
7. **Refactoring and testing:** remove duplication, improve constructor safety, add a regression test runner and document the final design.

4 Algorithm / Pseudocode and Flowchart

Pseudocode:

```
FUNCTION applyLeave(days, reason)
  validateRequest(days)
  balance = balance - days
  request = new LeaveRequest(days, reason)   // status = APPROVED
  save request in the employee's list
  RETURN request

FUNCTION validateRequest(days)
  IF days <= 0                  THEN throw InvalidLeaveRequestException
  IF days > maxDaysPerRequest() THEN throw InvalidLeaveRequestException
  IF days > balance             THEN throw InsufficientLeaveBalanceException

FUNCTION cancelLeave(requestId)
  FIND request with this id whose status is APPROVED
  IF found THEN mark CANCELLED and add its days back to balance
  ELSE throw InvalidLeaveRequestException
```

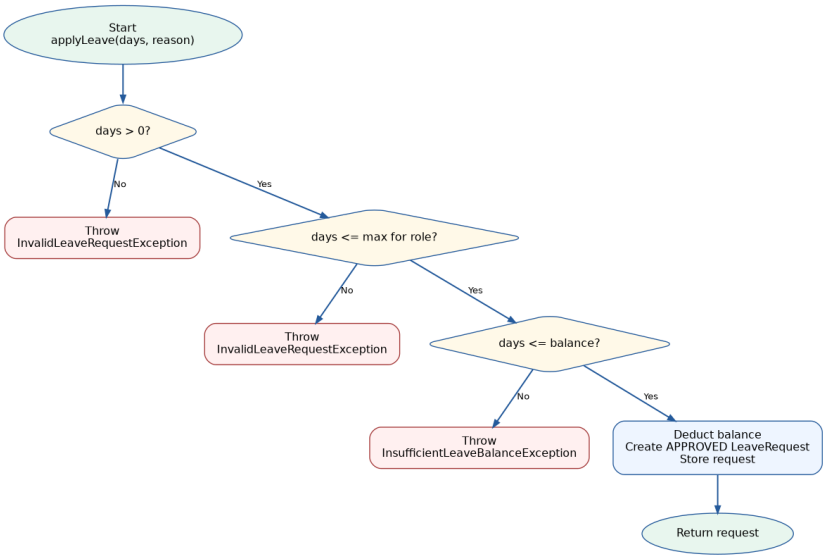


Figure 1: Final `applyLeave()` validation flowchart generated from the implemented design.

5 Implementation in Java: Class Diagram

The final implementation contains 12 Java source classes/interfaces: the OOP hierarchy, request model, exception hierarchy, main demonstration class and executable regression test class. Shared behaviour is kept in `Employee`, while role-specific rules are provided by subclasses.

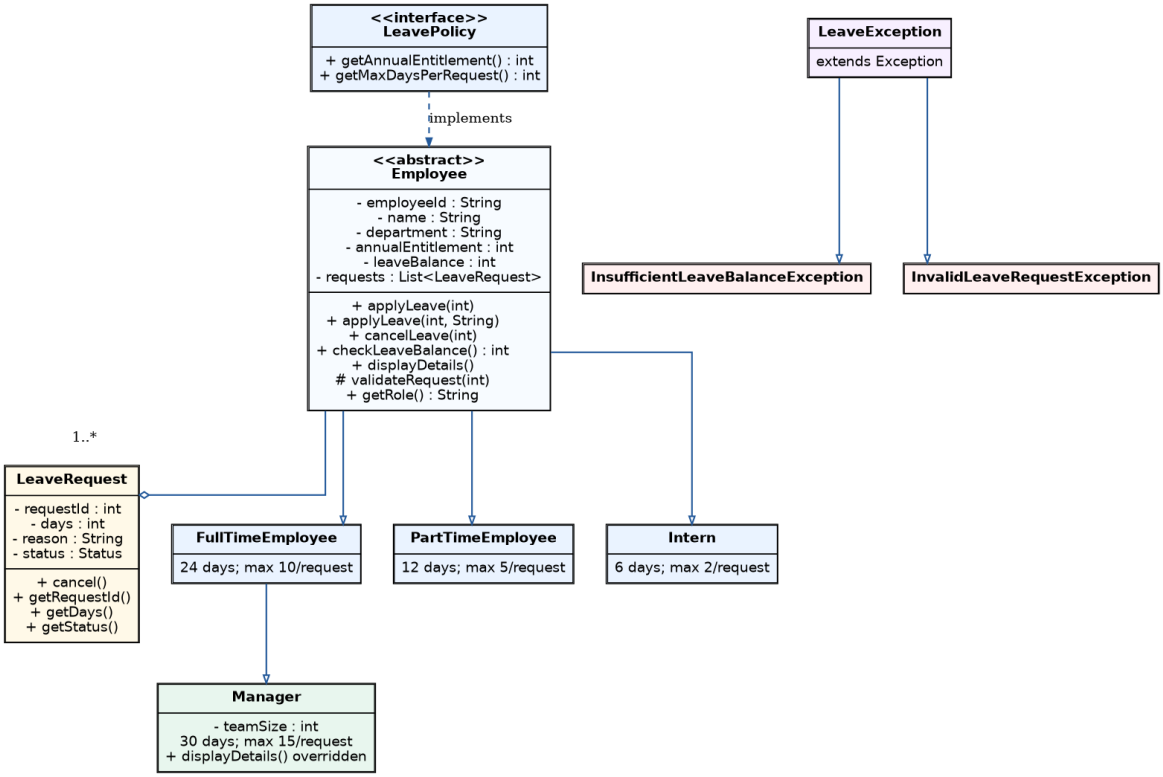


Figure 2: UML-style class diagram for the final source code.

Compile and run:

```
mkdir -p out
javac -d out src/leave/*.java
java -cp out leave.Main
```

Run the regression checks:

```
java -cp out leave.LeaveSystemSelfTest
```

6 Explanation of OOP Concepts Used

Concept	Where and how it is used
Encapsulation	Employee state is private. The balance changes only through controlled operations such as `applyLeave()` and `cancelLeave()`; callers read it using `checkLeaveBalance()`. `LeaveRequest` also protects its fields.
Abstraction	`Employee` is abstract and `LeavePolicy` is an interface. They define the common contract without putting every employee-specific rule in one large conditional block.
Inheritance	`FullTimeEmployee`, `PartTimeEmployee` and `Intern` extend `Employee`; `Manager` extends `FullTimeEmployee`.
Polymorphism	`Main` stores different concrete employee objects in a single `Employee[]` array and invokes the same methods on each object.
Method overriding	Employee subclasses provide their own maximum-per-request rule and role name. `Manager` overrides `displayDetails()` to add team size.
Method overloading	`applyLeave(int days)` delegates to `applyLeave(int days, String reason)` with a default reason.
Access modifiers	Fields are `private`; constructors and reusable validation hooks use `protected`; public methods form the external API.

7 Change Request: Adding Manager

The required change is a Manager employee type with additional leave benefits. `Manager` extends `FullTimeEmployee`, so the existing hierarchy and common operations are reused. The final design gives Manager a 30-day annual entitlement and a 15-day per-request limit and adds `teamSize` to its details.

A key refactoring was made while implementing this change: the annual entitlement is now supplied to the base constructor rather than being obtained by calling an overridable method from the `Employee` constructor. This removes a constructor-time polymorphism hazard while still allowing Manager to reuse the Full-Time hierarchy through a protected extension constructor.

8 Debugging and Refactoring Notes

#	**Improvement**	**Before / risk**	**Final implementation**
1	Duplicate validation	Days, request-limit and balance checks could be repeated in every employee subclass	One `validateRequest()` method in `Employee`; subclasses supply role-specific limits
2	Class responsibilities	Request ID, days, reason and status would be mixed into employee logic	Extracted `LeaveRequest` as a dedicated class
3	Exception handling	Invalid operations could be represented only by print messages or boolean results	Added `LeaveException`, `InvalidLeaveRequestException` and `InsufficientLeaveBalanceException`
4	Constructor safety	Calling an overridable entitlement method from the base constructor creates a fragile initialization dependency	Entitlement is supplied explicitly to the base constructor; Manager uses a protected FullTime constructor
5	Encapsulation of collections	Returning the internal request list directly would allow outside modification	`getRequests()` returns a defensive copy
6	Regression confidence	Manual checks alone are easy to repeat incorrectly	Added `LeaveSystemSelfTest` with executable regression assertions covering the assignment scenarios

9 Testing with Different Inputs

The assignment's testing requirement is covered through 13 scenario groups. The executable regression runner expands these into 15 assertions and reports the result automatically. The final run produced **15/15 regression checks passed**.

ID	Scenario	Expected / observed behaviour	Result
T1	Initial balances for Full-Time, Part-Time, Intern and Manager	24, 12, 6 and 30	Pass
T2	Full-Time applies 10 days	Approved; balance becomes 14	Pass
T3	Full-Time applies 11 days	Rejected because maximum is 10	Pass
T4	Part-Time applies 5 days	Approved; balance becomes 7	Pass
T5	Part-Time applies 6 days	Rejected because maximum is 5	Pass
T6	Intern applies 2 days	Approved; balance becomes 4	Pass
T7	Intern applies 3 days	Rejected because maximum is 2	Pass
T8	Manager applies 15 days	Approved; balance becomes 15	Pass
T9	Apply 0 days	Rejected: days must be positive	Pass
T10	Apply −3 days	Rejected: days must be positive	Pass
T11	Full-Time balance case	First valid request(s) reduce balance; an over-balance request is rejected	Pass
T12	Intern applies and cancels leave	Cancellation changes status and restores the balance	Pass
T13	Cancel a request ID that does not exist	Rejected: no active request with that ID	Pass

hline

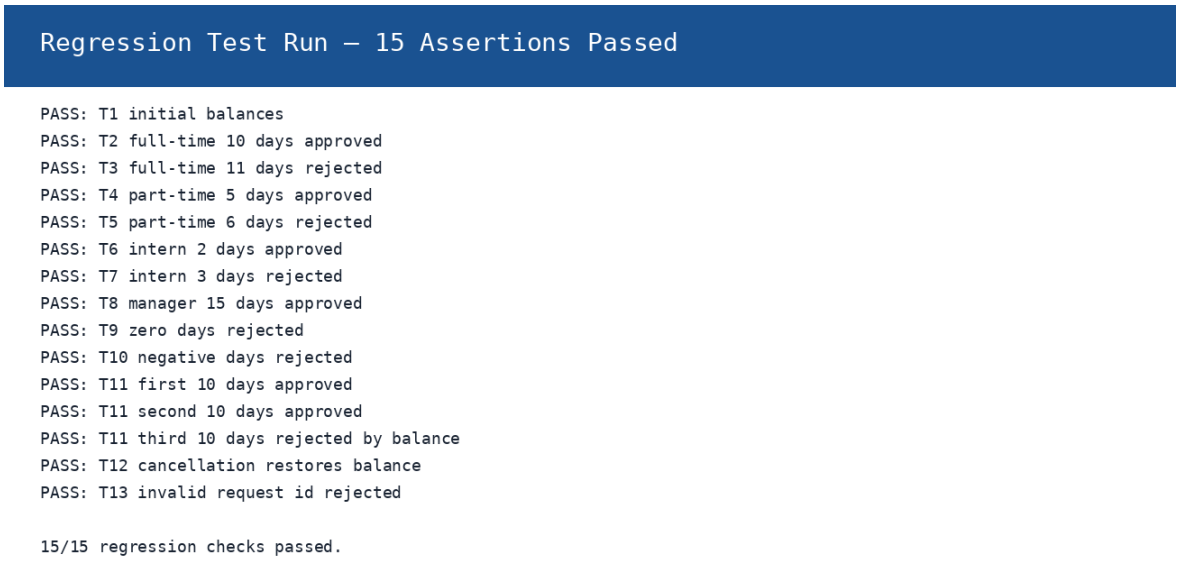


Figure 3: Actual regression test runner output: 15 assertions passed.

10 Sample Console Output

The following image is generated from the actual compiled `Main.java` execution included in this package.

Leave Management System — Sample Console Output

```
=== Initial details ===
[Full-Time] E001 | Asha | Engineering | Balance: 24/24 | Max/request: 10
[Part-Time] E002 | Ben | Design | Balance: 12/12 | Max/request: 5
[Intern] E003 | Chloe | Marketing | Balance: 6/6 | Max/request: 2
[Manager] E004 | Dev | Engineering | Balance: 30/30 | Max/request: 15
    Team size: 8

=== Applying leave ===
Asha -> Request#1 [4 day(s), Vacation, APPROVED] | balance 20
Ben -> Request#2 [4 day(s), Vacation, APPROVED] | balance 8
Chloe -> DENIED: Intern can take at most 2 day(s) per request.
Dev -> Request#3 [4 day(s), Vacation, APPROVED] | balance 26

=== Overloaded applyLeave(days) ===
Request#4 [2 day(s), Personal, APPROVED]

=== Cancel leave ===
Asha cancelled request 1, balance 22
Error: No active request with id 99

=== Exceeding balance ===
Dev -> DENIED: Requested 15 day(s) but only 11 available.

=== Final details ===
[Full-Time] E001 | Asha | Engineering | Balance: 22/24 | Max/request: 10
[Part-Time] E002 | Ben | Design | Balance: 8/12 | Max/request: 5
[Intern] E003 | Chloe | Marketing | Balance: 6/6 | Max/request: 2
[Manager] E004 | Dev | Engineering | Balance: 11/30 | Max/request: 15
    Team size: 8
```

Figure 4: Actual sample console output from the final Java program.

11 Challenges Faced

- **Manager entitlement design:** the earlier approach relied on an overridden entitlement method. The final version removes this constructor-time dependency and initializes the exact 30-day entitlement directly.
- **Where to put employee rules:** a large if-else chain inside `Employee` would need modification whenever a new employee type is added. Polymorphism keeps each role's rule with that role.
- **Cancellation safety:** cancelling the same request twice must not refund leave twice. Each `LeaveRequest` has a status, and only an APPROVED request can be cancelled.
- **Keeping responsibilities separate:** request data is now represented by `LeaveRequest`, while employee objects manage balances and operations.
- **Regression confidence:** the executable self-test was added so the core rules can be checked again after future changes.

12 Git Practice

The assignment asks for meaningful Git commits. The recommended history for this submission is:

```
Initial employee design and class diagram
Add Employee base class and leave exceptions
Implement leave request and cancellation flow
Add employee inheritance hierarchy
Implement polymorphism and method overloading
Add Manager change request
Refactor validation and request responsibilities
Add tests, README, diagrams and final report
```

Repository field: `https://github.com/<YOUR-USERNAME>/<YOUR-REPOSITORY>`

13 README File (README.md)

The final repository contains a README with the project objective, features, leave rules, OOP concepts, project structure, compilation instructions and author/repository fields. A copy is also included at the project root as `README.md`.

```
# Leave Management System (Java OOP)

Monthly Assignment - Module 2: Object-Oriented Programming

## Features
- Employee details and leave balance
- applyLeave() and cancelLeave()
- Full-Time, Part-Time, Intern and Manager types
- Custom exceptions
- Separate LeaveRequest class
- Regression test runner

## Leave Rules
Full-Time: 24 annual days, 10 maximum per request
Part-Time: 12 annual days, 5 maximum per request
Intern: 6 annual days, 2 maximum per request
Manager: 30 annual days, 15 maximum per request

## OOP Concepts
```

```
Encapsulation, Abstraction, Inheritance, Polymorphism,
Method Overriding, Method Overloading and Access Modifiers.

## Run
mkdir -p out
javac -d out src/leave/*.java
java -cp out leave.Main
java -cp out leave.LeaveSystemSelfTest
```

14 Possible Improvements

- Add dates to leave requests and prevent overlapping leave.
- Add leave categories such as sick, casual and paid leave.
- Add a manager approval workflow.
- Persist employees and requests to a file or database.
- Replace the executable self-test with JUnit tests in a larger production-style project.
- Add a menu-driven console or GUI front end.

15 Conclusion

The final project applies the core OOP concepts requested by the assignment to a realistic leave-management problem. Encapsulation protects employee and request state, abstraction defines the common employee contract, inheritance reuses the employee hierarchy, and polymorphism lets the same code operate on different employee types. Method overloading demonstrates multiple ways to apply leave, while overriding provides role-specific behaviour.

The Manager change request is implemented without duplicating the common leave-management logic. Refactoring also removes constructor-time polymorphism, separates request responsibilities, centralizes validation and protects the internal request list. The compiled application runs successfully, and the executable regression runner reports 15/15 assertions passed across the 13 assignment scenario groups.

A Java Source Code

LeavePolicy.java

```
package leave;

/** Abstraction: every employee type must define its leave rules. */
public interface LeavePolicy {
    int getAnnualEntitlement();
    int getMaxDaysPerRequest();
}
```

LeaveException.java

```
package leave;

/** Base exception for all leave-related errors. */
public class LeaveException extends Exception {
    private static final long serialVersionUID = 1L;
    public LeaveException(String message) {
        super(message);
    }
}
```

InsufficientLeaveBalanceException.java

```
package leave;

public class InsufficientLeaveBalanceException extends LeaveException {
    private static final long serialVersionUID = 1L;

    public InsufficientLeaveBalanceException(int requested, int available) {
        super("Requested " + requested + " day(s) but only " + available + " available.");
    }
}
```

InvalidLeaveRequestException.java

```
package leave;

public class InvalidLeaveRequestException extends LeaveException {
    private static final long serialVersionUID = 1L;

    public InvalidLeaveRequestException(String message) {
        super(message);
    }
}
```

LeaveRequest.java

```
package leave;

/** Separate class so Employee does not have to manage request details. */
public class LeaveRequest {
    public enum Status { APPROVED, CANCELLED }
```

```
    private static int counter = 1;
    private final int requestId;
    private final int days;
    private final String reason;
    private Status status;

    public LeaveRequest(int days, String reason) {
        this.requestId = counter++;
        this.days = days;
        this.reason = reason;
        this.status = Status.APPROVED;
    }

    public int getRequestId() { return requestId; }
    public int getDays() { return days; }
    public String getReason() { return reason; }
    public Status getStatus() { return status; }
    public void cancel() { this.status = Status.CANCELLED; }

    @Override
    public String toString() {
        return "Request#" + requestId + " [" + days + " day(s), " + reason + ", " + status + "]";
    }
}
```

Employee.java

```
package leave;

import java.util.ArrayList;
import java.util.List;

/** Abstract base class. Encapsulation: all fields are private. */
public abstract class Employee implements LeavePolicy {
    private final String employeeId;
    private final String name;
    private final String department;
    private final int annualEntitlement;
    private int leaveBalance;
    private final List<LeaveRequest> requests = new ArrayList<>();

    /**
     * The entitlement is supplied by the concrete class instead of calling an
     * overridable method from the constructor.
     */
    protected Employee(String employeeId, String name, String department, int annualEntitlement) {
        if (employeeId == null || employeeId.isBlank()) {
            throw new IllegalArgumentException("Employee ID cannot be blank.");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Employee name cannot be blank.");
        }
        if (department == null || department.isBlank()) {
            throw new IllegalArgumentException("Department cannot be blank.");
        }
        if (annualEntitlement <= 0) {
            throw new IllegalArgumentException("Annual entitlement must be positive.");
        }
        this.employeeId = employeeId;
        this.name = name;
        this.department = department;
        this.annualEntitlement = annualEntitlement;
        this.leaveBalance = annualEntitlement;
```

```
    }

    public String getEmployeeId() { return employeeId; }
    public String getName() { return name; }
    public String getDepartment() { return department; }
    @Override public int getAnnualEntitlement() { return annualEntitlement; }
    public int checkLeaveBalance() { return leaveBalance; }

    /** Overloading: apply with a default reason. */
    public LeaveRequest applyLeave(int days) throws LeaveException {
        return applyLeave(days, "Personal");
    }

    /** Overloading: apply with a reason. */
    public LeaveRequest applyLeave(int days, String reason) throws LeaveException {
        validateRequest(days);
        leaveBalance -= days;
        LeaveRequest request = new LeaveRequest(days, reason);
        requests.add(request);
        return request;
    }

    public void cancelLeave(int requestId) throws LeaveException {
        for (LeaveRequest request : requests) {
            if (request.getRequestId() == requestId && request.getStatus() == LeaveRequest.Status.
    APPROVED) {
                request.cancel();
                leaveBalance += request.getDays();
                return;
            }
        }
        throw new InvalidLeaveRequestException("No active request with id " + requestId);
    }

    /** Single place for all leave-request validation. */
    protected void validateRequest(int days) throws LeaveException {
        if (days <= 0) {
            throw new InvalidLeaveRequestException("Days must be positive.");
        }
        if (days > getMaxDaysPerRequest()) {
            throw new InvalidLeaveRequestException(
                    getRole() + " can take at most " + getMaxDaysPerRequest() + " day(s) per request.
    ");
        }
        if (days > leaveBalance) {
            throw new InsufficientLeaveBalanceException(days, leaveBalance);
        }
    }

    /** Subclasses supply their role name. */
    public abstract String getRole();

    /** Display common employee details; Manager extends this behaviour. */
    public void displayDetails() {
        System.out.println("[" + getRole() + "] " + employeeId + " | " + name + " | " + department
                + " | Balance: " + leaveBalance + "/" + getAnnualEntitlement()
                + " | Max/request: " + getMaxDaysPerRequest());
    }

    /** Defensive copy prevents callers from modifying the internal list. */
    public List<LeaveRequest> getRequests() {
        return new ArrayList<>(requests);
    }
}
```

FullTimeEmployee.java

```
package leave;

public class FullTimeEmployee extends Employee {
    public FullTimeEmployee(String id, String name, String dept) {
        this(id, name, dept, 24);
    }

    /** Protected extension point for subclasses such as Manager. */
    protected FullTimeEmployee(String id, String name, String dept, int annualEntitlement) {
        super(id, name, dept, annualEntitlement);
    }

    @Override public int getMaxDaysPerRequest() { return 10; }
    @Override public String getRole() { return "Full-Time"; }
}
```

PartTimeEmployee.java

```
package leave;

public class PartTimeEmployee extends Employee {
    public PartTimeEmployee(String id, String name, String dept) {
        super(id, name, dept, 12);
    }

    @Override public int getMaxDaysPerRequest() { return 5; }
    @Override public String getRole() { return "Part-Time"; }
}
```

Intern.java

```
package leave;

public class Intern extends Employee {
    public Intern(String id, String name, String dept) {
        super(id, name, dept, 6);
    }

    @Override public int getMaxDaysPerRequest() { return 2; }
    @Override public String getRole() { return "Intern"; }
}
```

Manager.java

```
package leave;

/** Change request: Manager reuses Full-Time structure with extra benefits. */
public class Manager extends FullTimeEmployee {
    private static final int MANAGER_ENTITLEMENT = 30;
    private final int teamSize;

    public Manager(String id, String name, String dept, int teamSize) {
        super(id, name, dept, MANAGER_ENTITLEMENT);
        if (teamSize < 0) {
            throw new IllegalArgumentException("Team size cannot be negative.");
        }
        this.teamSize = teamSize;
```

```
    }

    /** Protected constructor lets Manager reuse the Full-Time hierarchy safely. */
    protected Manager(String id, String name, String dept, int teamSize, int entitlement) {
        super(id, name, dept, entitlement);
        if (teamSize < 0) {
            throw new IllegalArgumentException("Team size cannot be negative.");
        }
        this.teamSize = teamSize;
    }

    @Override public int getMaxDaysPerRequest() { return 15; }
    @Override public String getRole() { return "Manager"; }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("    Team size: " + teamSize);
    }
}
```

Main.java

```
package leave;

public class Main {
    public static void main(String[] args) {
        // Polymorphism: one array type, many behaviours.
        Employee[] staff = {
            new FullTimeEmployee("E001", "Asha", "Engineering"),
            new PartTimeEmployee("E002", "Ben", "Design"),
            new Intern("E003", "Chloe", "Marketing"),
            new Manager("E004", "Dev", "Engineering", 8)
        };

        System.out.println("=== Initial details ===");
        for (Employee e : staff) e.displayDetails();

        System.out.println("\n=== Applying leave ===");
        for (Employee e : staff) {
            try {
                LeaveRequest r = e.applyLeave(4, "Vacation");
                System.out.println(e.getName() + " -> " + r + " | balance " + e.checkLeaveBalance());
            } catch (LeaveException ex) {
                System.out.println(e.getName() + " -> DENIED: " + ex.getMessage());
            }
        }

        System.out.println("\n=== Overloaded applyLeave(days) ===");
        try {
            System.out.println(staff[0].applyLeave(2));
        } catch (LeaveException ex) {
            System.out.println(ex.getMessage());
        }

        System.out.println("\n=== Cancel leave ===");
        try {
            staff[0].cancelLeave(1);
            System.out.println("Asha cancelled request 1, balance " + staff[0].checkLeaveBalance());
            staff[0].cancelLeave(99);
        } catch (LeaveException ex) {
            System.out.println("Error: " + ex.getMessage());
        }
```

```
        System.out.println("\n=== Exceeding balance ===");
        try {
            staff[3].applyLeave(15);
            staff[3].applyLeave(15);
        } catch (LeaveException ex) {
            System.out.println("Dev -> DENIED: " + ex.getMessage());
        }

        System.out.println("\n=== Final details ===");
        for (Employee e : staff) e.displayDetails();
    }
}
```

LeaveSystemSelfTest.java

```
package leave;

/** Lightweight executable regression tests for the assignment scenarios. */
public class LeaveSystemSelfTest {
    private static int passed = 0;
    private static int total = 0;

    private static void check(String name, boolean condition) {
        total++;
        if (!condition) throw new AssertionError("FAILED: " + name);
        passed++;
        System.out.println("PASS: " + name);
    }

    private static void expectLeaveException(String name, RunnableLeave operation, String
    expectedPart) {
        total++;
        try {
            operation.run();
            throw new AssertionError("FAILED: " + name + " (no exception)");
        } catch (LeaveException ex) {
            if (!ex.getMessage().contains(expectedPart)) {
                throw new AssertionError("FAILED: " + name + " -> " + ex.getMessage());
            }
            passed++;
            System.out.println("PASS: " + name);
        }
    }

    @FunctionalInterface
    private interface RunnableLeave {
        void run() throws LeaveException;
    }

    public static void main(String[] args) throws Exception {
        FullTimeEmployee ft = new FullTimeEmployee("T001", "FT", "Eng");
        PartTimeEmployee pt = new PartTimeEmployee("T002", "PT", "Design");
        Intern intern = new Intern("T003", "IN", "Marketing");
        Manager manager = new Manager("T004", "MG", "Eng", 8);

        check("T1 initial balances", ft.checkLeaveBalance() == 24 && pt.checkLeaveBalance() == 12
                && intern.checkLeaveBalance() == 6 && manager.checkLeaveBalance() == 30);

        LeaveRequest ft10 = ft.applyLeave(10, "Vacation");
        check("T2 full-time 10 days approved", ft10.getStatus() == LeaveRequest.Status.APPROVED
                && ft.checkLeaveBalance() == 14);
        expectLeaveException("T3 full-time 11 days rejected", () -> ft.applyLeave(11), "at most 10");

        pt.applyLeave(5, "Vacation");
```

```
        check("T4 part-time 5 days approved", pt.checkLeaveBalance() == 7);
        expectLeaveException("T5 part-time 6 days rejected", () -> pt.applyLeave(6), "at most 5");

        intern.applyLeave(2, "Training");
        check("T6 intern 2 days approved", intern.checkLeaveBalance() == 4);
        expectLeaveException("T7 intern 3 days rejected", () -> intern.applyLeave(3), "at most 2");

        manager.applyLeave(15, "Vacation");
        check("T8 manager 15 days approved", manager.checkLeaveBalance() == 15);

        expectLeaveException("T9 zero days rejected", () -> ft.applyLeave(0), "positive");
        expectLeaveException("T10 negative days rejected", () -> ft.applyLeave(-3), "positive");

        FullTimeEmployee balanceCase = new FullTimeEmployee("T005", "Balance", "Eng");
        balanceCase.applyLeave(10);
        check("T11 first 10 days approved", balanceCase.checkLeaveBalance() == 14);
        balanceCase.applyLeave(10);
        check("T11 second 10 days approved", balanceCase.checkLeaveBalance() == 4);
        expectLeaveException("T11 third 10 days rejected by balance", () -> balanceCase.applyLeave
    (10), "only 4");

        Intern cancelCase = new Intern("T006", "Cancel", "Marketing");
        LeaveRequest cancelRequest = cancelCase.applyLeave(1);
        cancelCase.cancelLeave(cancelRequest.getRequestId());
        check("T12 cancellation restores balance", cancelCase.checkLeaveBalance() == 6);

        expectLeaveException("T13 invalid request id rejected", () -> cancelCase.cancelLeave(999), "
    No active request");

        System.out.println("\n" + passed + "/" + total + " regression checks passed.");
    }
}
```